

Character LoRA Fine-tuning of a 33B Omni-modal Video Model

—System RAM, Not VRAM, Was the Binding Constraint—

Masato Suzuki (Masafy)

15 August 2026

Abstract

This report extends the author’s earlier work—character LoRA fine-tuning of Stable Diffusion 1.5 (about 1B parameters) [1] and of Krea 2 (about 12B) [2]—into video generation. The target is **MiniMax H3**, an omni-modal model that generates video and audio jointly, with a diffusion transformer of roughly 33B parameters paired with a 32B Qwen3-VL text encoder. The subject is the same original character (a red panda, “Masafy”) used in both earlier studies. Those two studies each concluded that a single consumer GPU with 12 GB of VRAM suffices; **here the same GPU did not suffice**. Tracing the failure by measurement showed that the binding constraint is not VRAM but **system RAM**. Peak VRAM during training measured 3,959 MiB—2.8% of the card actually used—while loading the 41.2 GB model set failed three times by OOM on hosts with 50 GB of system RAM and completed on a host with 188 GB. We further show that (a) the trained adapter transfers across quantization formats (trained on pruned INT8, applied to GGUF Q3_K_M, all 208 modules matching) and runs inference within 9.3 GB of VRAM; (b) with a reference-image conditioning scheme, the LoRA contributes *style* rather than structural stability; and (c) adapter strength trades style fidelity against structural accuracy, with a cross-validated optimum at 500 steps and strength 0.8. Total compute cost was about five US dollars.

Keywords: video generation, omni-modal models, diffusion transformer, low-rank adaptation, LoRA, system memory, quantization, consumer GPU, reproducibility

1 Introduction

In two earlier studies the author fine-tuned an original character—“Masafy”, a red panda—into Stable Diffusion 1.5 [1] and then Krea 2 [2] using low-rank adaptation (LoRA) [3], each time on a single consumer GPU with 12 GB of VRAM. The second study combined FP8 quantization with block swapping to bring a 12B model—roughly 24 GB of weights at half precision—inside a 12 GB card.

Together these results supported a general claim: *with the right techniques, personal fine-tuning of a current generative model fits on one consumer GPU*. This study asks whether the claim survives the move to **video generation**.

MiniMax H3 generates video and audio jointly. Its diffusion transformer holds roughly 33B parameters and it pairs with the 32B variant of the Qwen3-VL text encoder. It offers Reference-to-Video-Audio (R2V) conditioning, which pins a character from a reference image.

The research questions are:

- Q1.** On the same 12 GB GPU used before, can a 33B-class video model be LoRA-trained? If not, what is the real constraint?
- Q2.** Does the trained adapter transfer to an inference stack that uses a different quantization? What does inference cost?
- Q3.** When a reference image already pins the character, what does the LoRA add? Where are the best strength and training length?

The main contribution is a negative answer to Q1 together with the identification of its cause—the author marking the boundary of a claim his

Table 1: Model composition

Component	Size
Transformer (pruned INT8 ConvRot)	20.0 GB
Text encoder (Qwen3-VL 32B)	15.7 GB
VAE (video fp16 / audio fp32)	5.5 GB
Total	41.2 GB

own earlier work established.

2 Setup

2.1 Model composition

Table 1 lists the components. The text encoder alone is 32B; with the transformer, loading amounts to a 65B-class set of weights. That fact is the direct cause of the constraint reported below.

2.2 Training configuration

Training used ai-toolkit [8] pinned at commit f4e9130. The LoRA is rank 16 / alpha 16 over `attn.qkv_proj`, `attn.out_proj`, `mlp.fc1` and `mlp.fc2`, reaching **208 modules** (416 tensors) across 50 blocks plus the token refiner.

The dataset is 12 training images, 2 validation and 2 excluded, each 1000×1000, drawn by the author. Frame count is 1—stills treated as single-frame video—with batch size 1, gradient checkpointing on, bf16, at resolutions 512 and 768.

Every caption opens with `masafy_character`, followed by the fixed appearance (aviator goggles, pat-

terned scarf) and then a per-image description of pose, expression and background. Caption dropout is 0.05.

2.3 Environments

Four environments were tried (Table 2). Recording *both* VRAM and system RAM is the methodological point of this study.

3 Q1: Identifying the constraint

3.1 Failure on the local GPU

On the RTX 3060 (12 GB VRAM, 30 GB RAM) used in both earlier studies, a pilot run **completed zero steps in two hours and twenty-five minutes**. GPU utilization read 0.0% at every sample; the run halted just after **Quantizing transformer** and produced nothing.

Utilization pinned at zero means the GPU was not waiting on compute—the run never reached compute. The `low_vram` path offloads weights to the host, but 41.2 GB of weights against 30 GB of RAM pushes the remainder into swap.

The first lesson follows: **the existence of a `low_vram` option does not imply the run will work**. Check that system RAM can hold the whole model before checking VRAM at all.

3.2 Reproducing and isolating the failure

On 48 GB VRAM / 50 GB RAM the run died at the same point both times, killed by signal (exit 137):

```
Loading Qwen3-VL text encoder
- attached 351 pre-quantized layers
Text encoder is already quantized
Killed
```

After the 20.0 GB transformer, adding the 15.7 GB text encoder reaches 36 GB; runtime overhead carries it past the 50 GB ceiling.

Crucially, the second attempt with `low_vram disabled` failed **at the same point**. Weights are staged in host RAM before transfer to the device, so the failure is independent of free VRAM—which indeed measured 0–3 MiB at that moment.

Hosts with 188 GB and 233 GB both completed. The requirement therefore lies **above 50 GB and at or below 188 GB**.

Figure 2 plots VRAM and system RAM against outcome. Two 48 GB hosts sit on opposite sides of the outcome column, so VRAM carries no signal; system RAM separates cleanly between 50 and 188.

3.3 RAM allocation does not track price

Surveying commercial offerings shows that system RAM correlates with neither price nor VRAM (Table 3).

The RTX 4090 costs more than the 3090 and ships a third of its RAM. The original plan called for a 4090;

none was in stock. *Had one been available, the run would have failed earlier than it did on the A40.*

Allocations also vary within a single GPU model depending on the hosting tier. Third-party-hosted RTX 3090 instances measured 30 GB and 41 GB of RAM against an advertised 125 GB (two independent draws).

3.4 The successful run

On the RTX 6000 Ada (188 GB RAM, \$0.84/h), 800 steps took 9.04 s per step, stable to the last step, for a wall clock of 2h00m15s at \$1.89. Figure 3 shows the loss trace.

Setup time tracked vCPU count closely: 2,100 s on a 9-vCPU host versus 147–366 s on 14–24-vCPU hosts. **Costlier GPUs ship more vCPUs and set up faster**, so the expected penalty—that expensive hosts waste money during setup—did not materialize.

3.5 A failure mode worth naming

The package manager installs a PyTorch built for CUDA 13 by default, while the host driver supported CUDA 12.8. In that state training starts, reports progress and continues indefinitely, but runs on CPU: `torch.cuda.is_available()` returns false.

The fix is to pin `torch==2.13.0+cu126`. Note that **pinning the version number alone does nothing**: the same version is already present, and the package manager does not compare build tags. The local version identifier must be part of the specifier.

Because this failure presents as “running but not progressing,” it is externally indistinguishable from the swap-thrashing failure of Section 3.1. Any pipeline should therefore **assert `torch.cuda.is_available()` before training and refuse to start otherwise**.

4 Q2: Transfer across quantizations

Training ran against pruned INT8 safetensors; inference uses GGUF Q3_K_M. These are different quantizations of one model, and matching weight keys is not guaranteed.

Applying the adapter reported 208 patches **attached**—exactly the 50 blocks $\times$ 4 modules plus token refiner the LoRA targets—with zero key-mismatch warnings.

Different quantization formats do not block transfer as long as layer naming is preserved. Training and inference may therefore use different quantizations, which is a useful degree of freedom in practice.

Inference used 9,326 MiB of VRAM and took about 13 minutes for a 5-second clip at 864 $\times$ 480 on the RTX 3060. **Training required a host with 188 GB of system RAM; inference fits a consumer GPU**. That asymmetry is the practical takeaway: rent for training, run at home.

Training set: 12 images of the original character (1000x1000)



Figure 1: The training set: 12 images of the character drawn by the author. Most carry the die-cut border of their original sticker use.

5 Q3: What the LoRA contributes

5.1 Ablating the adapter

R2V pins the character from a reference image, which invites the question of whether a LoRA is needed at all. To answer it, only the presence of the adapter was varied: the reference image came from the validation set (unseen in training), and prompt, seed, resolution and sampler steps were held fixed.

Table 4 gives the result. Without the LoRA the character still renders without breakage, the accessories are correct, and no inter-frame flicker appears. **The style, however, is entirely different.**

The roles separate cleanly: *the reference image carries what is drawn; the LoRA carries how it is drawn.* “A reference image makes the LoRA unnecessary” is therefore false. The return on a character LoRA is not the ability to render the character—it is the ability to render it **in one’s own style**.

5.2 Strength and training length

Twenty-four clips spanning strength 0.0–1.2 and checkpoints 300–800 were generated under fixed conditions and evaluated qualitatively (Figures 5 and 6).

Style fidelity and structural accuracy move in opposite directions. Raising strength moves the style toward the target while degrading complex regions—hands above all. Training length plateaus near step 500; later steps add nothing, and step 700 showed mild degradation.

5.3 Cross-validation and the recommended setting

The two sweeps above were run at fixed values of the other axis—strength at step 800, checkpoints at strength 1.0—leaving the combination of the two best values untested. Generating strengths 0.6, 0.8 and 1.2 at checkpoint 500 closed that gap.

Checkpoint 500 at strength 0.8 was best overall. At 0.6 the style has not moved far enough. At 1.2 the style is closest to target, but a *heavy dark contour appears around the whole character*—a faithful repro-

Table 2: Environments and outcomes. VRAM does not predict success; system RAM does.

Environment	GPU	VRAM	System RAM	Price	Outcome
Local	RTX 3060	12 GB	30 GB	electricity	fail (0 steps in 2h25m)
Cloud A	A40	48 GB	50 GB	\$0.44/h	fail (OOM, <code>low_vram</code> on)
Cloud A'	A40	48 GB	50 GB	\$0.44/h	fail (OOM, <code>low_vram</code> off)
Cloud B	H200 SXM	141 GB	233 GB	\$4.59/h	success (pilot)
Cloud C	RTX 6000 Ada	48 GB	188 GB	\$0.84/h	success (full run)

VRAM does not predict success; system RAM does

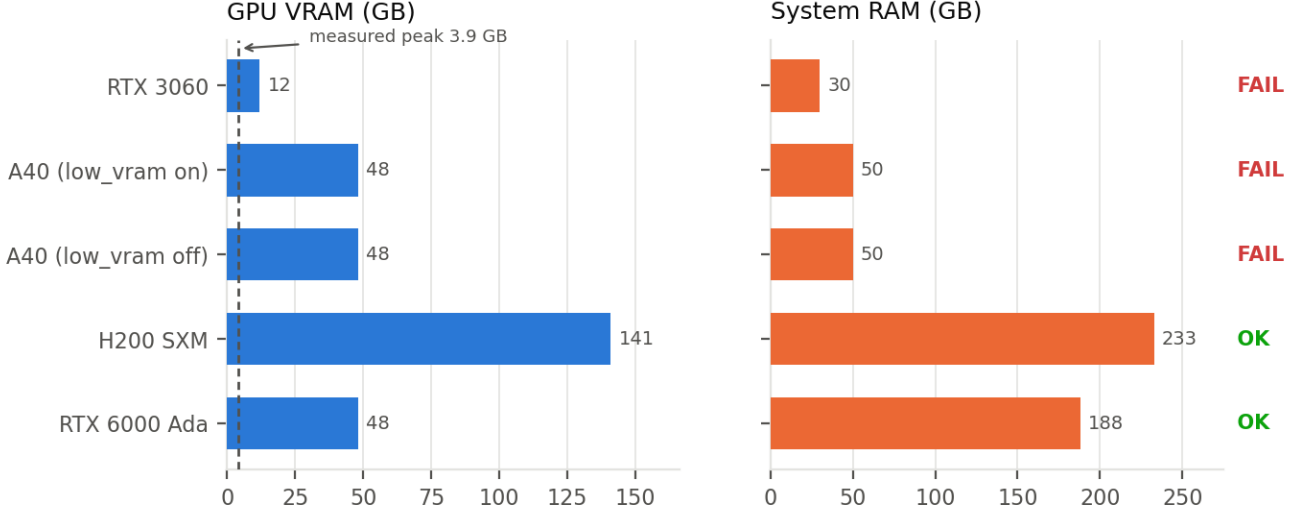


Figure 2: VRAM does not predict success; system RAM does. The dashed line marks measured peak VRAM during training (3.9 GB).

Table 3: Advertised host allocations

GPU	VRAM	System RAM	Price
RTX 3090	24 GB	125 GB	\$0.50/h
RTX 4090	24 GB	41 GB	\$0.69/h
RTX 6000 Ada	48 GB	167 GB	\$0.84/h
A40	48 GB	50 GB	\$0.44/h
H100 NVL	94 GB	94 GB	\$3.19/h

Table 4: With and without the adapter, all else fixed

Criterion	With	Without
Structural stability	good	good
Inter-frame flicker	none	none
Accessories	good	good
Style fidelity	good	entirely different
Output size	525 KB	1,087 KB

duction of the die-cut border in the sticker artwork used for training. Natural in a sticker, it reads as an artifact in video.

The recommendation is therefore **500 steps at strength 0.8**, and the initially chosen 800 steps at strength 1.0 was *excessive in both training and inference*. Training drops from about two hours to about 1.25, and cost from \$1.89 to roughly \$1.2.

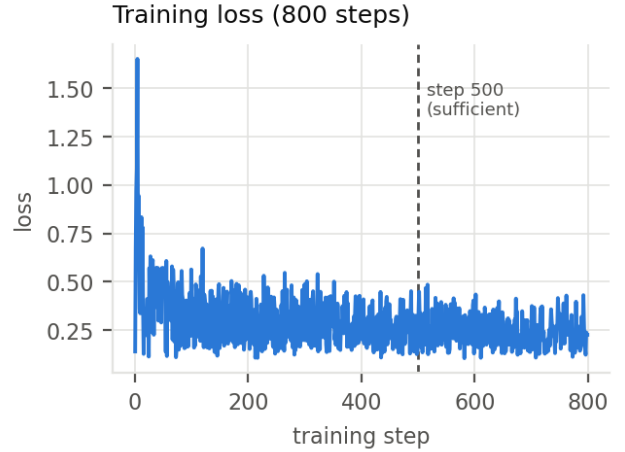


Figure 3: Training loss over 800 steps (799 points recorded).

5.4 What the dataset smuggles in

Cross-validation surfaced three ways the adapter carries properties of the training data that were never asked for.

First, the **contour** described above.

Second, **fine detail needs strength**. Captions read **pink patterned scarf**, yet at strengths 0.6 and 0.8 the scarf is nearly plain; the pattern appears only at

Same seed, same reference image; only the adapter differs

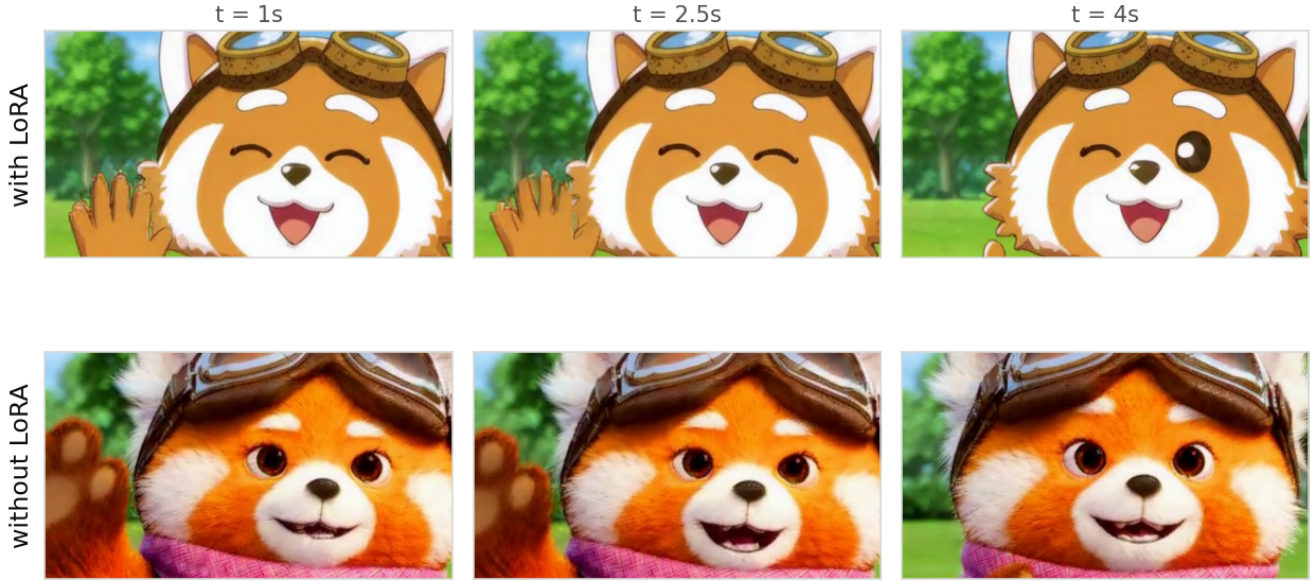


Figure 4: Same seed, same reference image, adapter toggled. The top row renders as flat illustration; the bottom row renders individual strands of fur.



Figure 5: Style shift against adapter strength. Output size serves as a proxy for how far the style has moved.

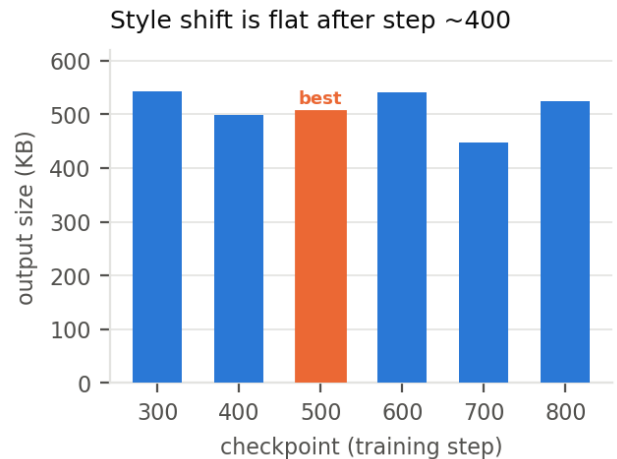


Figure 6: Effect of training length. Style shift plateaus after roughly step 400.

1.2.

Third, **the adapter reshapes backgrounds too**. A depth-blurred background at 0.6 becomes graphic linework at 1.2. Captions described whole images, so the learned style spans the frame—too broad a reach if the goal is to swap only the character.

Each of these shows *caption design and dataset provenance surfacing directly as properties of the output*.

5.5 The trigger token

Removing the trigger while keeping the appearance description preserved the elements but lost the style. Supplying the trigger alone, with no appearance description, failed to produce a usable result. **The adapter does not fully internalize the appearance; the caption**

text is doing real work. With only 12 training images, all captioned with the same appearance clause, this is the expected outcome.

5.6 On output size as a proxy

Output file size decreased monotonically with strength, from 1,087 KB at 0.0 to 492 KB at 1.2. The training art is flat-shaded, so as the style converges the tonal range narrows and the codec compresses better.

Size is nonetheless **not a quality metric**. The smallest outputs—strength 1.2 and checkpoint 700—were exactly the ones carrying structural breakage or artifacts. Size measures style displacement and nothing else.

Adapter strength at checkpoint 500

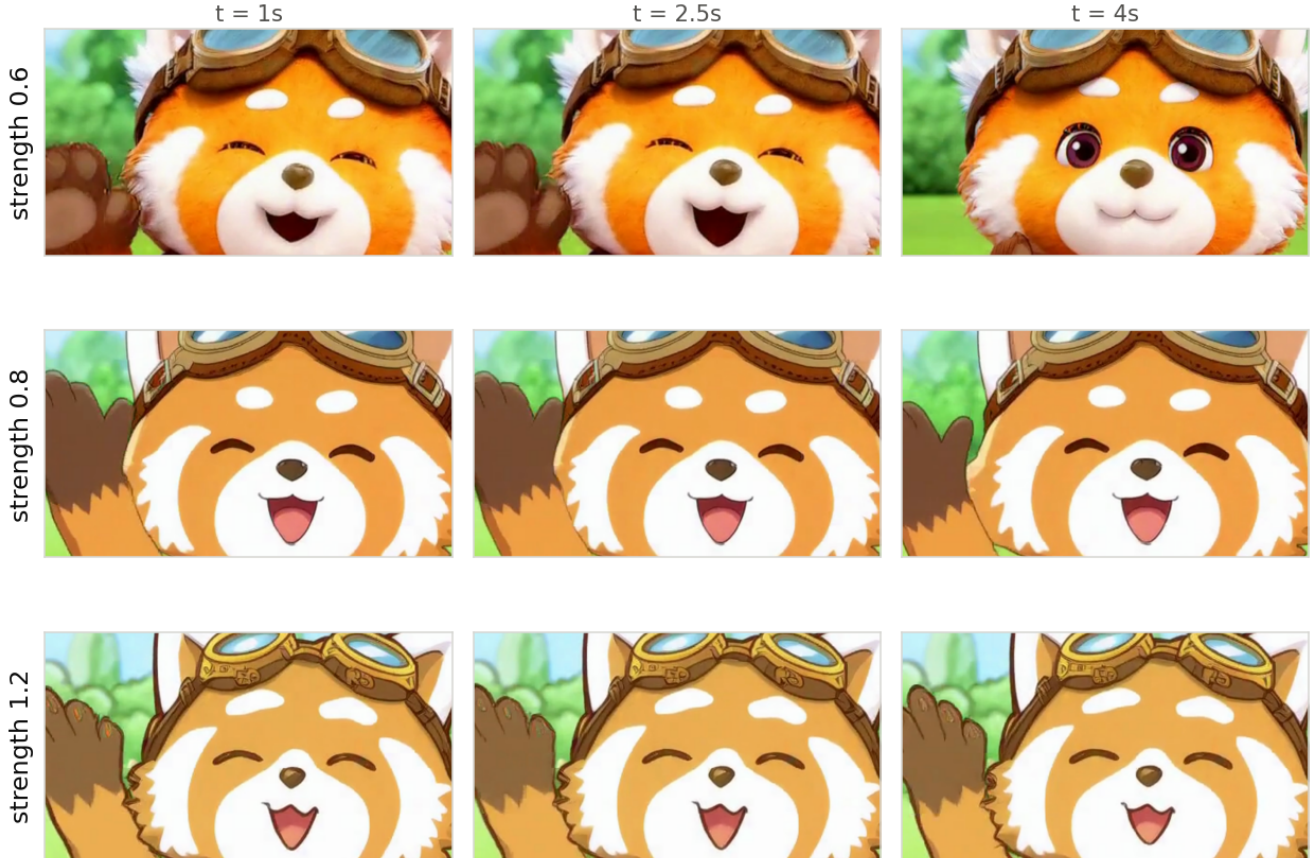


Figure 7: Adapter strength at checkpoint 500. At 1.2 a heavy dark contour wraps the whole character.

6 Limitations

1. **Peak system RAM during training was never measured.** The monitor recorded GPU statistics only. The requirement can therefore be stated as a range—above 50 GB, at or below 188 GB—and no more precisely. Instrumentation has since been added.
2. **Cost is approximate.** The account balance before the A40 attempts was not recorded, so the total is stated as about five dollars; the span measurable through the API was \$3.44.
3. **Qualitative evaluation came from a single rater** without blinding.
4. **The dataset holds 12 images.** Conclusions about the trigger token and about generalization may be specific to that scale.
5. **Generalization to unseen scenes was limited.** In settings absent from training—a neon street at night, riding a bicycle—the adapter degraded detail relative to the un-adapted baseline.

7 Conclusion

For LoRA training of a 33B omni-modal video model, the binding constraint was **system RAM**, not VRAM (Q1). Peak VRAM measured 3,959 MiB—2.8% of the installed card—while 50 GB of system RAM failed during loading and 188 GB completed.

The claim carried by the two earlier studies [1, 2]—that a single consumer GPU suffices—**does not hold at this scale**. But it fails for a reason other than the expected one: not too little VRAM, too little system RAM. What one should rent is therefore not a GPU with large VRAM but **a machine with large system RAM**, and because advertised RAM tracks neither price nor VRAM, that has to be checked deliberately.

The trained adapter transfers across quantization formats and infers within 9.3 GB of VRAM (Q2), so **training can be rented while operation stays on local consumer hardware**.

Under reference-image conditioning the adapter contributes style rather than structural stability (Q3). Strength trades style fidelity against structural accuracy, and cross-validation puts the optimum at **500 steps, strength 0.8**. The initial choice was excessive.

Total compute cost was about five US dollars.

Reproducibility

Scripts, dataset, measurement logs and the source of this paper are published at:

- <https://github.com/masafykun/minimax-h3-masafy-lora>
- <https://huggingface.co/masafy/minimax-h3-masafy-lora>

References

- [1] M. Suzuki. Character LoRA Fine-tuning for Image Generation: An Experimental Study under Few-shot Conditions on a Consumer GPU. Technical report, 2026.
- [2] M. Suzuki. Character LoRA Fine-tuning of a 12B Image Generation Model: FP8 Quantization and Block Swapping on a Single Consumer GPU, with Transfer to a Distilled Model. Technical report, 2026. DOI: 10.5281/zenodo.20838898.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations (ICLR)*, 2022.
- [4] W. Peebles and S. Xie. Scalable Diffusion Models with Transformers. *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [5] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow Matching for Generative Modeling. *International Conference on Learning Representations (ICLR)*, 2023.
- [6] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman. DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [7] Qwen Team. Qwen-Image Technical Report. 2025.
- [8] ostris. AI Toolkit. <https://github.com/ostris/ai-toolkit>, 2026.
- [9] comfyanonymous. ComfyUI. <https://github.com/comfyanonymous/ComfyUI>, 2026.